

JavaScript Essentials

Understanding the Fundamentals of
JavaScript

What is JavaScript?

- Definition:
 - JavaScript is a versatile programming language for web development. It allows creating dynamic and interactive web content.
- Key Features:
 - Runs in browsers
 - Event-driven programming
 - Platform-independent
- Example:
 - `<button onclick="alert('Welcome to JavaScript!')">Click Me!</button>`

The Birth of JavaScript (1995)

- Created by **Brendan Eich** in 1995
- Developed at **Netscape Communications**
- Originally called *Mocha*, then *LiveScript*
- Renamed **JavaScript** for marketing reasons
- Designed to make web pages interactive

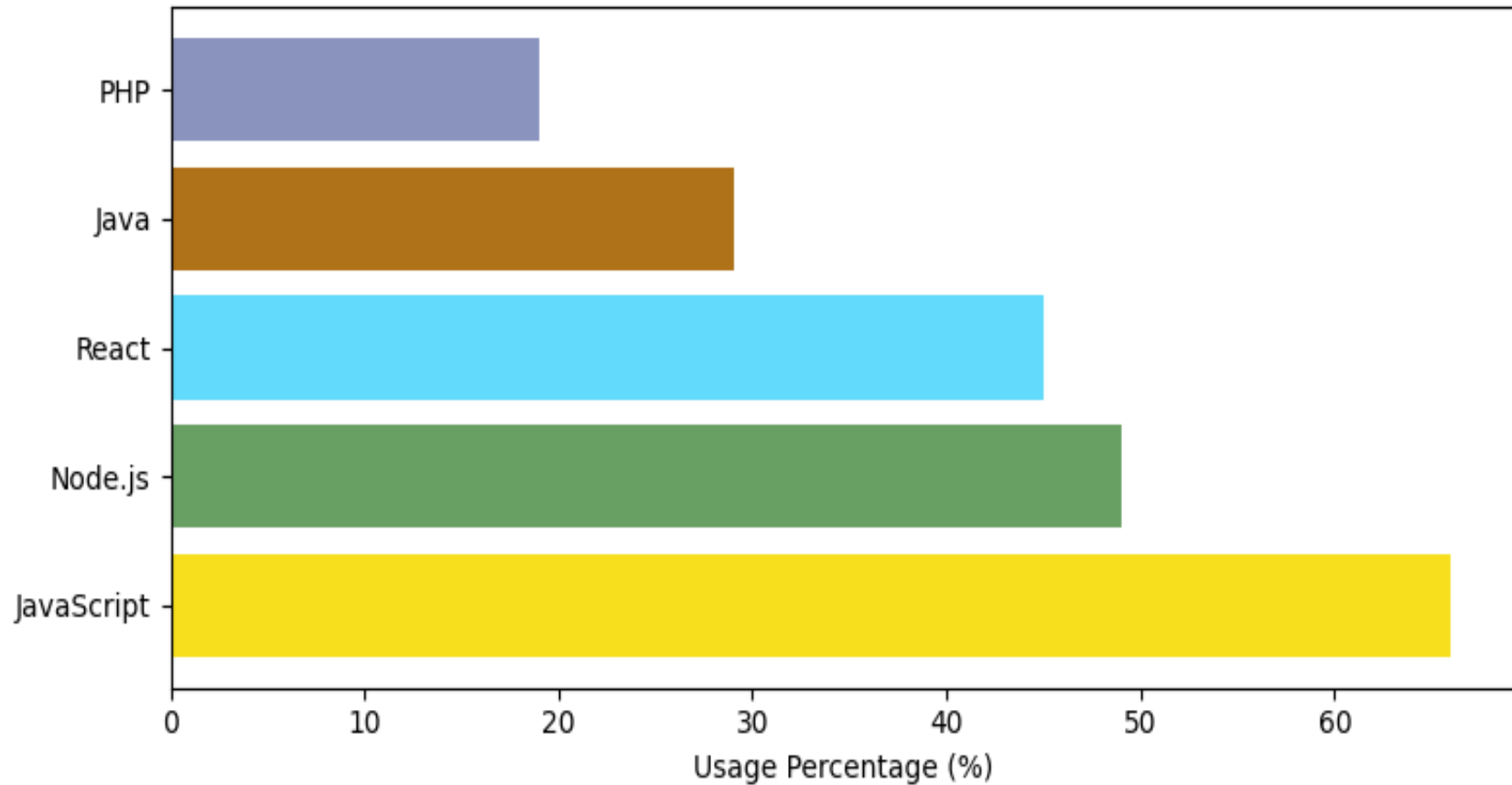
Standardization & Growth (1997–2009)

- Standardized as **ECMA International** (ECMAScript – 1997)
- Widely supported by browsers like **Internet Explorer**
- 2009: Launch of **Node.js**
- JavaScript moves beyond browsers (server-side development)

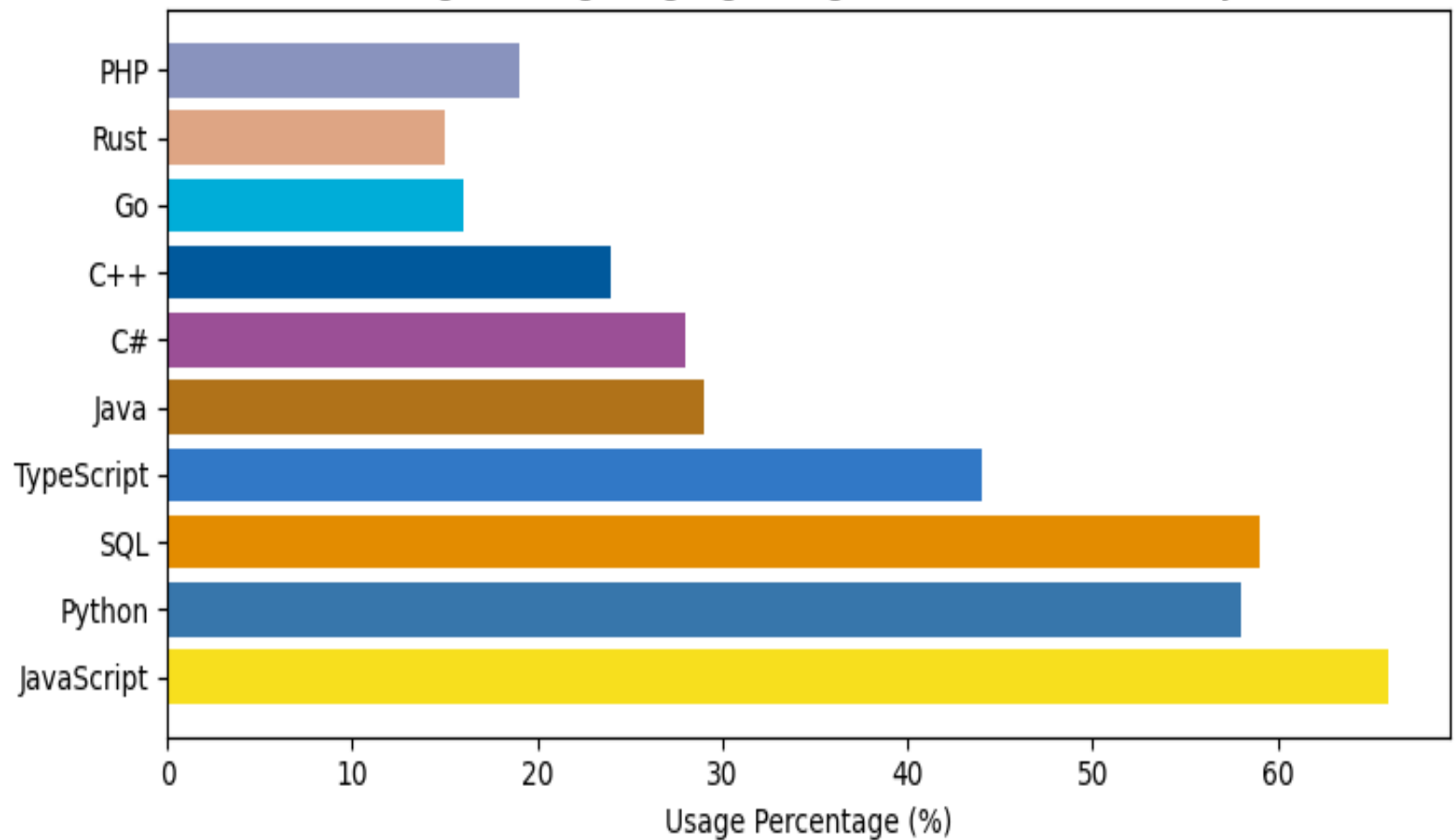
Modern JavaScript (2015–Today)

- 2015: Major update (ES6 / ECMAScript 2015)
- Introduction of let, const, arrow functions, classes
- Rise of frameworks like **React**, **Angular**, and **Vue.js**
- Now one of the most popular programming languages worldwide

Full-Stack Ecosystem Usage (Stack Overflow Survey 2025)



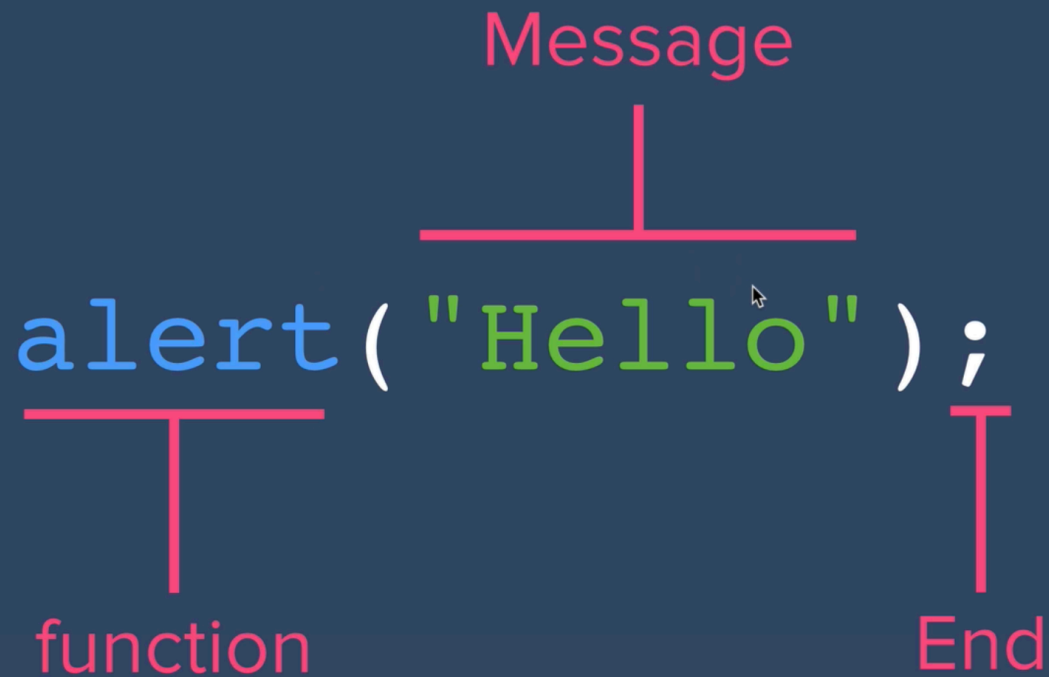
Global Programming Language Usage (Stack Overflow Survey 2025)



JavaScript Alert - Adding Behavior to a Website

- Types of Alerts:
 - alert(): Simple message box
 - confirm(): OK/Cancel dialog box
 - prompt(): Input dialog box
- Examples:
 - alert('Hello!');
 - let response = prompt('What is your name?');
 - alert('Welcome, ' + response + '!');

JavaScript Alert

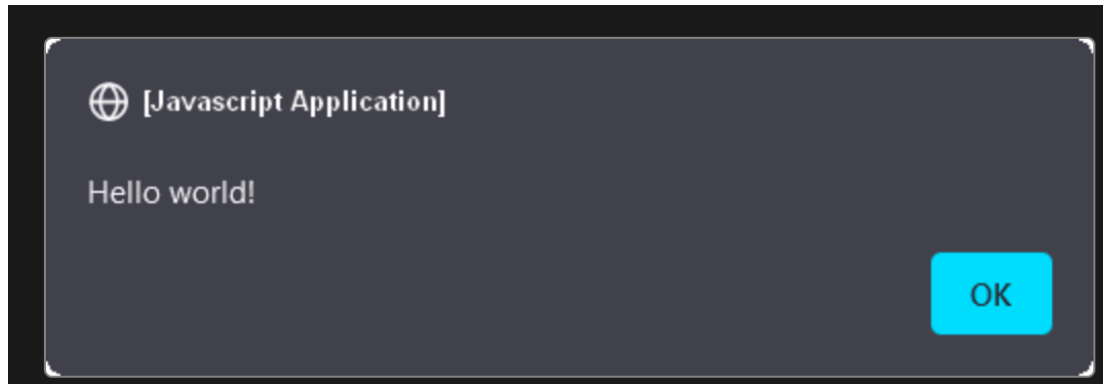


The diagram illustrates the syntax of the JavaScript `alert` function. It features the code `alert("Hello");` on a dark blue background. The word `alert` is in blue, the string `"Hello"` is in green, and the semicolon `;` is in white. Three red annotations are present: a horizontal line with a vertical line pointing down to the word `function` below it; a horizontal line with a vertical line pointing up to the word `Message` above it; and a horizontal line with a vertical line pointing up to the semicolon `;` below it, labeled `End`.

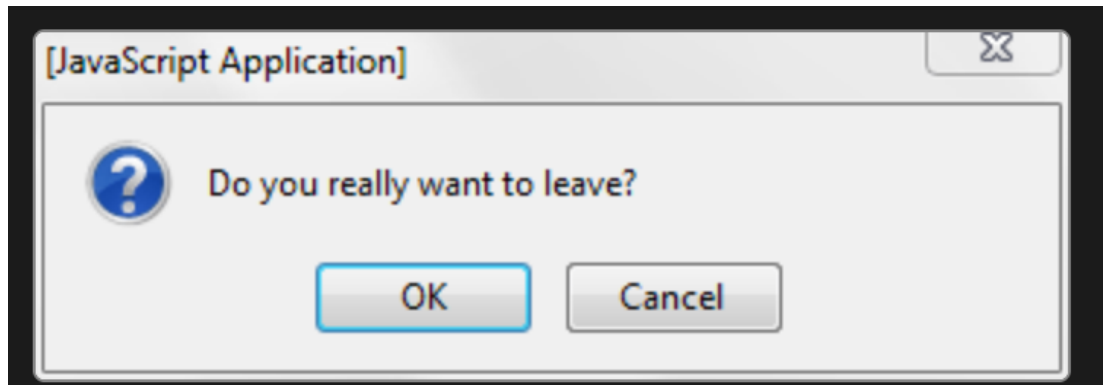
```
function alert(Message);
```

JavaScript Alert

- Alert

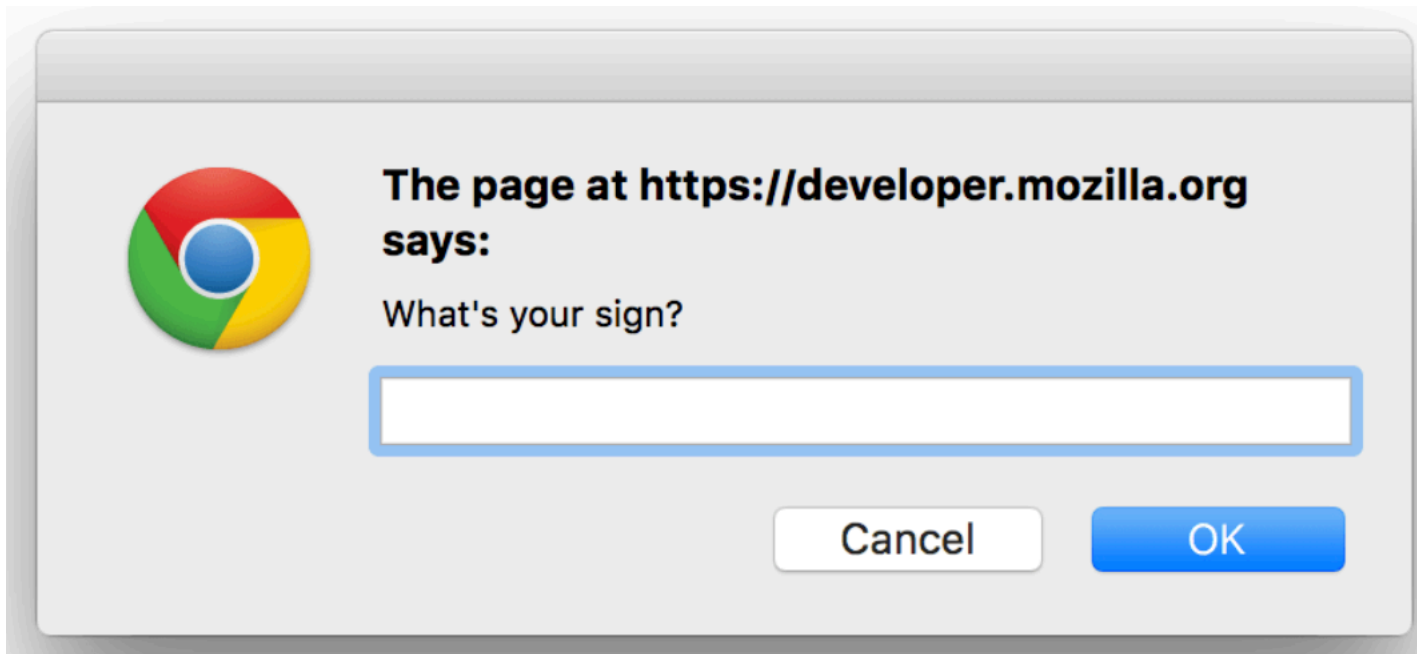


- Confirm



JavaScript Alert

- Prompt



JavaScript Data Types

- Common Data Types:
 - Numbers, Strings, Booleans, Null, Undefined, Objects
- Examples:
 - `let age = 25;      // Number`
 - `let name = 'John';   // String`
 - `let isAdmin = true;   // Boolean`
 - `let value = null;    // Null`
- Type Checking:
 - `console.log(typeof age); // 'number'`

JavaScript Variables

- In JavaScript, **variables** are containers used to store data values
- Variables allow you to label and reuse data in your programs
- JavaScript provides three ways to declare variables:
 - var: Function-scoped
 - let: Block-scoped
 - const: Block-scoped (constants)
- Syntax:

javascript

 Copy code

```
var variableName = value;  
let variableName = value;  
const variableName = value;
```

Using var

- var is **function-scoped**, meaning it is limited to the function where it is declared.
- It does **not respect block scope** (e.g., if or for blocks).
- Can be **redeclared** within the same scope.

javascript

 Copy code

```
var name = "Alice";
console.log(name); // Output: Alice

if (true) {
  var name = "Bob";
  console.log(name); // Output: Bob
}

console.log(name); // Output: Bob (var ignores block scope)
```

Using let

- let is **block-scoped**, meaning it is limited to the block where it is declared (e.g., inside {}).
- Cannot be redeclared within the same scope, but the value can be reassigned.

javascript

 Copy code

```
let age = 25;
console.log(age); // Output: 25

if (true) {
  let age = 30; // Different variable (block-scoped)
  console.log(age); // Output: 30
}
console.log(age); // Output: 25 (original `age` is unaffected)
```

Using const

- `const` is **block-scoped**, like `let`.
- Cannot be **redeclared** or **reassigned**.
- Requires a value to be initialized during declaration.

javascript

 Copy code

```
const PI = 3.14159;  
console.log(PI); // Output: 3.14159  
  
// PI = 3.14; // Error: Assignment to constant variable
```


Naming Conventions for Variables

- **Variable Naming Rules**

- Meaningful name
- Must begin with a letter, _ (underscore), or \$ (dollar sign).
- Cannot start with a number.
- Can contain letters, numbers, _, and \$.
- **Case-sensitive** (e.g., myVar and MyVar are different).
- Reserved keywords (like class, return) cannot be used
- No space between to part of a name

- **Examples:**

- let userName; // Good
- let \$price = 100; // Good
- let 1number = 10; // Invalid

String Concatenation

- Joining Strings:
 - `let firstName = 'John';`
 - `let lastName = 'Doe';`
 - `let fullName = firstName + ' ' + lastName; // John Doe`
- Template Literals:
 - `console.log(`Hello, ${firstName} ${lastName}`);`

String Length and Extracting Characters

- Length:
 - `let text = 'JavaScript';`
 - `console.log(text.length); // 10`
- Access Characters:
 - `console.log(text[0]); // J`
 - `console.log(text[text.length - 1]); // t`

Slicing and Extracting Part of a String

- Methods:
 - `slice(start, end)`: Extracts part of the string
 - `substring(start, end)`: Similar, but no negative indices
- Example:
 - `let str = 'JavaScript is fun!';`
 - `console.log(str.slice(0, 10)); // JavaScript`

Changing String Casing

- Methods:
 - `toUpperCase()`: Converts to uppercase
 - `toLowerCase()`: Converts to lowercase
- Example:
 - `let text = 'JavaScript';`
 - `console.log(text.toUpperCase()); // JAVASCRIPT`
 - `console.log(text.toLowerCase()); // javascript`

Basic Arithmetic and Modulo Operator

- Arithmetic Operators:
 - Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulo (%)
- Example:
 - `let num1 = 10;`
 - `let num2 = 3;`
 - `console.log(num1 + num2); // 13`
 - `console.log(num1 % num2); // 1`

Examples of Arithmetic Precedence

- 1. Parentheses First
- 2. Exponentiation Precedes Multiplication
- 3. Division and Multiplication (Left to Right)
- 4. Modulo Operation
- 5. Combined Example

Examples of Arithmetic Precedence

- Parentheses First

javascript

 Copy code

```
let result = (5 + 3) * 2;  
console.log(result); // Output: 16
```

Without parentheses:

javascript

 Copy code

```
let result = 5 + 3 * 2;  
console.log(result); // Output: 11
```



Examples of Arithmetic Precedence

- 2. Exponentiation Precedes Multiplication

javascript

 Copy code

```
let result = 2 * 3 ** 2;  
console.log(result); // Output: 18
```

Explanation:

`3 ** 2` is evaluated first (exponentiation), resulting in `9`. Then, `2 * 9` equals `18`.

Examples of Arithmetic Precedence

- 3. Division and Multiplication (Left to Right)

javascript

 Copy code

```
let result = 20 / 5 * 2;  
console.log(result); // Output: 8
```

- Explanation:
Division and multiplication are evaluated left to right:
 $20 / 5 = 4$, then $4 * 2 = 8$.

Examples of Arithmetic Precedence

- 4. Modulo Operation

javascript

 Copy code

```
let result = 10 % 3 + 2;  
console.log(result); // Output: 3
```

- Explanation:

$10 \% 3 = 1$ (remainder), then $1 + 2 = 3$.

Examples of Arithmetic Precedence

- 5. Combined Example with Multiple Operators

javascript

 Copy code

```
let result = 5 + 10 * 3 - 4 / 2 ** 2;  
console.log(result); // Output: 33
```

- Step-by-step evaluation:
- $2 ** 2 = 4$
- $4 / 4 = 1$
- $10 * 3 = 30$
- $5 + 30 - 1 = 33$

Increment and Decrement

- Operators:
 - Increment (++): Adds 1 to a variable
 - $X = x + 1 \rightarrow x++$
 - Decrement (--): Subtracts 1 from a variable
 - $X = x - 1 \rightarrow x--$

Increment and Decrement

- JavaScript has two types of increment operators:
 - **Pre-increment (++variable)**: The variable is incremented *before* being used in the expression.
 - **Post-increment (variable++)**: The variable is incremented *after* being used in the expression.

Pre-Increment Example (++variable)

- Increases the value of the variable *before* it is used.

javascript

 Copy code

```
let x = 5;  
let y = ++x; // x is incremented first, then its value is assigned to  
  
console.log(x); // Output: 6 (x is incremented)  
console.log(y); // Output: 6 (y gets the incremented value of x)
```

Post-Increment Example (variable++)

- The variable's current value is used first, and then it is incremented.

javascript

 Copy code

```
let a = 5;  
let b = a++; // a is used first, then incremented  
  
console.log(a); // Output: 6 (a is incremented)  
console.log(b); // Output: 5 (b gets the original value of a)
```


Assignment Operator

- Adds the right-hand value to the left-hand variable and assigns the result to the variable.

javascript

 Copy code

```
let num = 10;
```

```
num += 5; // num = num + 5; -> num becomes 15
```

```
num -= 3; // num = num - 3; -> num becomes 12
```

```
num *= 2; // num = num * 2; -> num becomes 24
```

```
num /= 4; // num = num / 4; -> num becomes 6
```

```
console.log(num); // Output: 6
```

Exercises

- 1. Extract the first 10 characters of a string.
 - `let quote = 'JavaScript is awesome!';`
 - `console.log(quote.slice(0, 10));`
- 2. Calculate sum, product, and remainder:
 - `let a = 15, b = 4;`
 - `console.log(a + b, a * b, a % b);`
- 3. Prompt user for input and display a greeting:
 - `let name = prompt('Enter your name:');`
 - `alert(`Hello, ${name}!`);`

Conclusion

- Recap:
 - JavaScript basics: Variables, Strings, and Operators
 - Exercises to practice
- Further Learning:
 - MDN Web Docs: developer.mozilla.org